

Analisar o seu GitHub (pauloskaterock) sob a ótica de um recrutador ou gerente de engenharia do **Google (para a vaga de SRE)** é o passo mais realista que podemos dar agora. O Google usa o seu código público para validar se você apenas "sabe a teoria" ou se realmente **constrói software com mentalidade de engenharia**.

Como um SRE no Google lida com código que roda em escala global, o seu perfil no GitHub precisa refletir **organização, automação real (via código, não cliques) e boas práticas de desenvolvimento**.

Aqui está uma análise diagnóstica detalhada do que o seu perfil precisa transmitir nos próximos 12 meses, dividida entre a **Apresentação**, o **Código** e a **Evolução SRE**:

1. A Vitrine: O Perfil Geral (Readme)

O primeiro impacto visual conta muito. O recrutador do Google passa, em média, menos de 30 segundos olhando um perfil antes de decidir abrir um repositório.

- **O que você já tem de bom:** O seu link do portfólio (softwarephengineer.netlify.app) diretamente na bio é excelente. Mostra capricho e que você se posiciona profissionalmente.
- **O que precisamos ajustar para SRE:** O seu Readme principal (aquele que aparece logo de cara no perfil) deve ser o seu "cartão de visitas de infraestrutura".
 - **Dica Prática:** Crie um repositório com o mesmo nome do seu usuário (pauloskaterock). O GitHub transforma isso em um Readme especial na sua página inicial.
 - **O que colocar lá:** Adicione um resumo dinâmico focado em automação. Coloque suas certificações (OCI, AWS) em formato de *badges* (distintivos visuais) e liste as tecnologias que você domina (Python, Linux/WSL, Bash, Ruby).

2. A Anatomia dos Repositórios (O que o Google avalia)

Quando o time técnico do Google abre um repositório seu, eles olham três pilares fundamentais:

- **Histórico de Commits (Mensagens claras):** Evite commits com mensagens genéricas como "update", "teste" ou "arrumando bug". O Google valoriza o padrão *Conventional Commits* (ex: feat: add log parser for nginx, fix: handle memory leak exception in routine). Isso demonstra maturidade profissional.
- **Documentação (O arquivo README.md de cada projeto):** Um projeto sem um bom Readme é um projeto invisível. Cada automação Python ou sistema Ruby que você subir deve conter:
 1. O que o projeto faz (focado no problema de infraestrutura que ele resolve).
 2. Como instalar as dependências (pip install -r requirements.txt).
 3. Como executar o projeto no terminal (de preferência via comandos Linux/WSL).
- **Organização de Arquivos:** Manter o código limpo, usando ambientes virtuais (.venv) e garantindo que arquivos de configuração sensíveis (como senhas ou chaves de API) estejam no .gitignore e nunca expostos publicamente.

3. Plano de Transformação do seu GitHub para SRE

(Próximos Passos)

Para que o seu GitHub vire um "ímã de vagas" de SRE no Google nos próximos meses, precisamos equilibrar os seus projetos antigos (RPA e Ruby) com **projetos puros de automação de sistemas**.

Aqui está a trilha de repositórios que você deve construir e deixar pinados (em destaque) no seu perfil:



Desafio Prático para iniciar hoje no seu GitHub:

Aproveitando que você está fazendo o curso da Udemy e manipula dados, crie um repositório chamado **system-health-monitor**.

1. Escreva um script simples em Python que roda no seu WSL, lê o uso de espaço em disco do sistema e salva o resultado em um arquivo .csv (usando o que você está aprendendo sobre arquivos e tabelas).
2. Se o uso do disco passar de 80%, faça o script imprimir um alerta vermelho no terminal.
3. Suba esse código para o GitHub com um README.md bem estruturado explicando como rodar o script no Linux.

Isso vai mostrar para o Google que você sabe usar Python para **garantir a saúde e a confiabilidade de um servidor**, que é exatamente a definição do papel de um SRE.

Gostou da ideia do desafio para começarmos a movimentar o seu GitHub com foco em infraestrutura? Se quiser, podemos estruturar a lógica do código desse monitor juntos aqui!